1. What is a Vector?

A vector is a dynamic array from the C++ Standard Template Library (STL) that:

- Can **grow or shrink** in size.
- Handles memory management automatically.
- Offers **random access** like arrays.

```
#include <vector>
using namespace std;

vector<int> v;  // Empty vector of integers
```

2. Basic Operations

Operation	Code Example	Description
Add element	v.push_back(10);	Adds 10 at the end
Remove last element	v.pop_back();	Removes the last element
Access element	v[i] or v.at(i)	Random access
Size of vector	v.size();	Returns current size
Check if empty	v.empty();	Returns true if vector is empty
Last element	v.back();	Returns the last element
First element	v.front();	Returns the first element

3. Vector Initialization Examples

```
vector<int> v1;              // empty
vector<int> v2(5);           // [0, 0, 0, 0, 0]
vector<int> v3(5, 2);        // [2, 2, 2, 2, 2]
vector<int> v4 = {1, 2, 3, 4};   // initializer list
vector<int> v5(v4);          // copy constructor
```

4. Iterating Over a Vector

```
for (int i = 0; i < v.size(); i++)
    cout << v[i] << " ";
```

```
for (int x : v) // range-based
    cout << x << " ";


for (auto it = v.begin(); it != v.end(); it++)
    cout << *it << " ";
```

5. Common Functions

```
v.clear();          // remove all elements
v.erase(v.begin()); // remove first element
v.insert(v.begin() + 2, 99); // insert 99 at index 2
sort(v.begin(), v.end()); // sort
reverse(v.begin(), v.end()); // reverse
```

6. Behind the Scenes (Memory)

- Vectors have a **capacity** (allocated size) and **size** (actual elements).
- When size exceeds capacity, a new bigger array is allocated, old data is copied, and the new value is inserted.

Vector – Key Properties and Time Complexities

Operation	Time Complexity	Notes
Access (indexing)	O(1)	Like arrays, random access is allowed
Insertion at End	O(1) *amortized*	Occasionally O(n) when resizing occurs
Insertion at Middle/Front	O(n)	All elements right of insertion must be shifted
Deletion from End	O(1)	Like pop_back()
Deletion from Middle/Front	O(n)	All elements right of deletion point must be shifted
Search (linear)	O(n)	Use loop or algorithms like find()

Operation	Time Complexity	Notes
Dynamic Resizing	O(n)	Occurs when size exceeds capacity; reallocates and copies data
Size Check	O(1)	Via .size()
Memory Usage	More than arrays	Allocates extra memory to handle future insertions efficiently

---

- **Additional Notes:**
  - Vectors **store elements in contiguous memory**, just like arrays.
  - The **capacity** usually doubles when reallocation happens — this keeps average insertions efficient.
  - Use v.reserve(n) if you know in advance how many elements you'll need — it prevents repeated reallocations.